AI ASSISTENT LAB - 21.3

Baisa Vaishnavi
2303A52142
Batch - 41

Task 1 – AI-Assisted Pair Programming

Example:

Use AI to generate an initial Expense Tracker Application.

• Ask AI to create a basic program to record daily expenses and categories.

• Prompt AI to suggest improvements such as:

o Monthly expense summary

o Input validation

o Code refactoring into functions

• Compare the initial and improved versions to understand AI's role in collaborative coding.

Initial prompt:

Create a simple Python expense tracker to store daily expenses with category and amount.

Code:

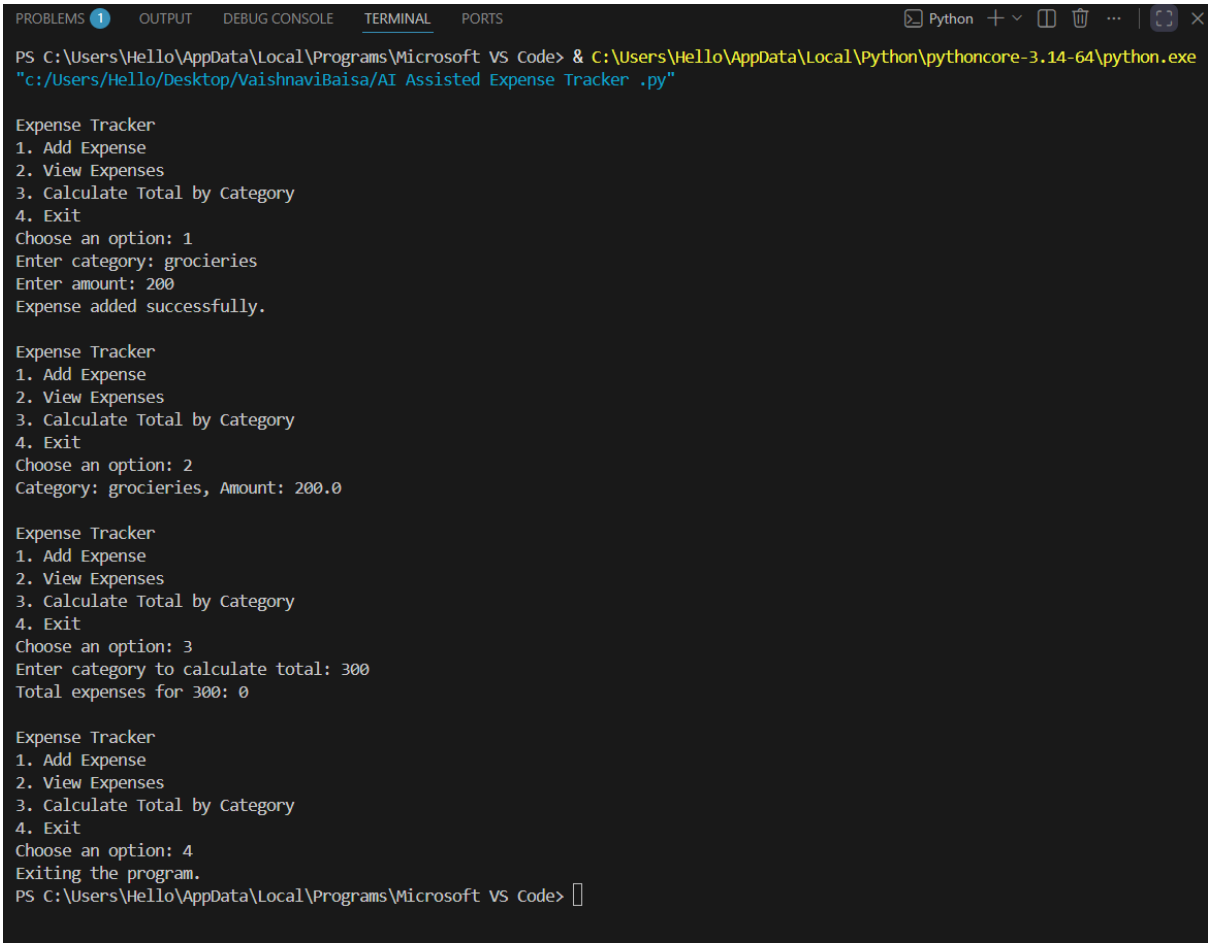
AI Assisted Expense Tracker .py

C: > Users > Hello > Desktop > VaishnaviBaisa > AI Assisted Expense Tracker .py > ...

```python
#Create a simple Python expense tracker to store daily expenses with category and amount.
#The program should allow users to add new expenses, view all expenses, and calculate the total expens
class ExpenseTracker:
    def __init__(self):
        self.expenses = []

    def add_expense(self, category, amount):
        self.expenses.append({'category': category, 'amount': amount})

    def view_expenses(self):
        for expense in self.expenses:
            print(f"Category: {expense['category']}, Amount: {expense['amount']}")

    def calculate_total_by_category(self, category):
        total = sum(expense['amount'] for expense in self.expenses if expense['category'] == category)
        return total
def main():
    tracker = ExpenseTracker()
    while True:
        print("\nExpense Tracker")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Calculate Total by Category")
        print("4. Exit")
        choice = input("Choose an option: ")

        if choice == '1':
            category = input("Enter category: ")
            amount = float(input("Enter amount: "))
            tracker.add_expense(category, amount)
            print("Expense added successfully.")
        elif choice == '2':
            tracker.view_expenses()
        elif choice == '3':
```

```python
def main():
        elif choice == '3':
            category = input("Enter category to calculate total: ")
            total = tracker.calculate_total_by_category(category)
            print(f"Total expenses for {category}: {total}")
        elif choice == '4':
            print("Exiting the program.")
            break
        else:
            print("Invalid option. Please try again.")
if __name__ == "__main__":    main()
```

Output:



Justification:

- Functions → modular design
- Validation → avoids runtime errors
- Summary → real-world usability

Task 2 – Version Control Integration

Example:

Manage a Student Grade Calculator project using Git.

• Initialize a Git repository for the project.

• Create a new branch to add AI-assisted features such as:

o Grade classification (A, B, C, etc.)

o Average and percentage calculation

• Commit changes with meaningful commit messages.

• Merge the feature branch back into the main branch and resolve conflicts if any.

AI Prompt:

Add grade classification and percentage calculation to the program.

Code:

```
C: > Users > Hello > Desktop > VaishnaviBaisa > Student Grade Calculator.py > ...
1   #Add grade classification and percentage calculation to the program.
2   def calculate_grade(score):
3       if score >= 90:
4           return 'A', score * 0.1
5       elif score >= 80:
6           return 'B', score * 0.1
7       elif score >= 70:
8           return 'C', score * 0.1
9       elif score >= 60:
10          return 'D', score * 0.1
11      else:
12          return 'F', score * 0.1
13  # Example usage
14  score = float(input("Enter the score: "))
15  grade, percentage = calculate_grade(score)
16  print(f"Grade: {grade}, Percentage: {percentage:.2f}%")
```

Output:

```
PROBLEMS 1  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                Python

PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/Hello/Desktop/VaishnaviBaisa/Student Grade Calculator.py"
Enter the score: 70
Grade: C, Percentage: 7.00%
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/Hello/Desktop/VaishnaviBaisa/Student Grade Calculator.py"
Enter the score: 80
Grade: B, Percentage: 8.00%
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/Hello/Desktop/VaishnaviBaisa/Student Grade Calculator.py"
Enter the score: 90
Grade: A, Percentage: 9.00%
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/Hello/Desktop/VaishnaviBaisa/Student Grade Calculator.py"
Enter the score: 99
Grade: A, Percentage: 9.90%
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/Hello/Desktop/VaishnaviBaisa/Student Grade Calculator.py"
Enter the score: 30
Grade: F, Percentage: 3.00%
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/Hello/Desktop/VaishnaviBaisa/Student Grade Calculator.py"
Enter the score: 20
Grade: F, Percentage: 2.00%
PS C:\Users\Hello\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

- Branching → safe feature development
- Merge → integration workflow
- AI → faster feature addition

Task 3 – AI for Commit Message Generation

Example:

Use AI to generate commit messages for a Password Validation Module.

• Modify the code to add rules like:

o Minimum length
o Special character requirement
• Ask AI to generate commit messages describing these updates.
• Compare AI-generated messages with human-written ones and analyze clarity.

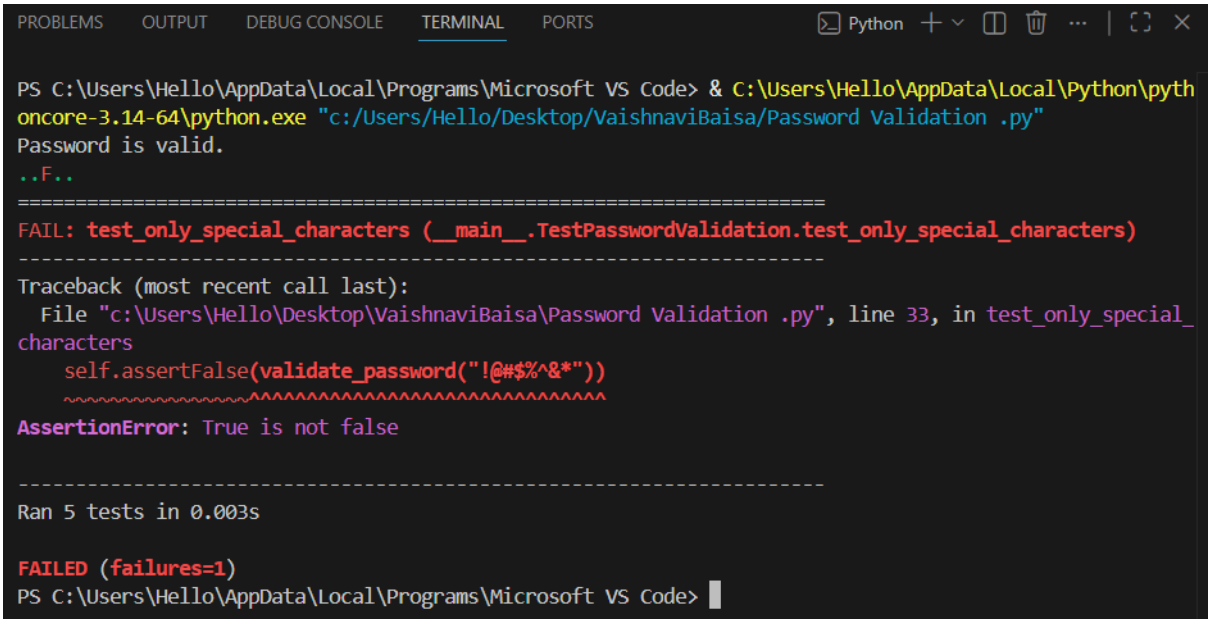
Prompt:
Generate commit message for adding password validation with length and special character rules.

Code:

C: > Users > Hello > Desktop > VaishnaviBaisa > Password Validation .py > ...

```python
#Generate commit message for adding password validation with length and special
#generate the code
import re

def validate_password(password):
    if len(password) < 8:
        return False
    if not re.search(r'[!@#$%^&*()-+]', password):
        return False
    return True
# Example usage
password = "P@ssw0rd"
if validate_password(password):
    print("Password is valid.")
else:
    print("Password is invalid. It must be at least 8 characters long and contai
# Unit tests
import unittest
class TestPasswordValidation(unittest.TestCase):
    def test_valid_password(self):
        self.assertTrue(validate_password("P@ssw0rd"))

    def test_short_password(self):
        self.assertFalse(validate_password("P@ss1"))

    def test_no_special_character(self):
        self.assertFalse(validate_password("Password123"))

    def test_empty_password(self):
        self.assertFalse(validate_password(""))

    def test_only_special_characters(self):
        self.assertFalse(validate_password("!@#$%^&*"))
if __name__ == '__main__':
    unittest.main()
```

Output:
AI Commit Message



Human Version
Added strong password validation rules

Justification
● AI → more structured (uses “feat:” convention)
● Human → less descriptive

Task 4 – Pair Programming Simulation
Scenario:
Collaboratively develop a Quiz Management System.
• Student A creates the initial quiz program with questions and answers.
• Student B, with AI assistance, adds features such as:
o Score calculation
o Input validation
• Both students collaborate using Git branching and merging to integrate their changes.

Prompt:
Add score calculation and input validation to quiz program.

Code:

```
"""
STUDENT A - INITIAL QUIZ MANAGEMENT SYSTEM
==========================================

This is the initial version of the Quiz Management System created by Student A.
It includes:
- Basic quiz structure with questions and answers
- Simple quiz navigation
- Basic display functionality

Features Added by Student A:
✓ Quiz data structure
✓ Question and answer management
✓ Basic quiz flow
✓ Question display
✓ Answer input
✓ Final score calculation and display

TODO (For Student B to enhance):
- Input validation
- Enhanced error handling
- Performance metrics
- Statistical analysis
"""
```

```python
def create_quiz():
    """
    Create a quiz with questions and answers.
    Returns a dictionary containing quiz data.
    """
    quiz = {
        "title": "Python Fundamentals Quiz",
        "description": "Test your knowledge of Python programming basics",
        "questions": [
            {
                "id": 1,
                "question": "What is the output of print(2 ** 3)?",
                "options": ["6", "8", "2", "1"],
                "correct_answer": "8",
                "explanation": "2 ** 3 means 2 to the power of 3, which equals 8
            },
            {
                "id": 2,
                "question": "Which data type is immutable in Python?",
                "options": ["list", "set", "tuple", "dictionary"],
                "correct_answer": "tuple",
                "explanation": "Tuples are immutable and cannot be modified afte
            },
            {
                "id": 3,
                "question": "What will len([1, 2, 3, 4]) return?",
                "options": ["3", "4", "5", "2"],
                "correct_answer": "4",
                "explanation": "The list has 4 elements, so len() returns 4"
            },
            {
                "id": 4,
                "question": "How do you create a function in Python?",
                "options": ["function myFunc():", "def myFunc():", "func myFunc(
                "correct_answer": "def myFunc():",
                "explanation": "The 'def' keyword is used to define a function i
            },
```

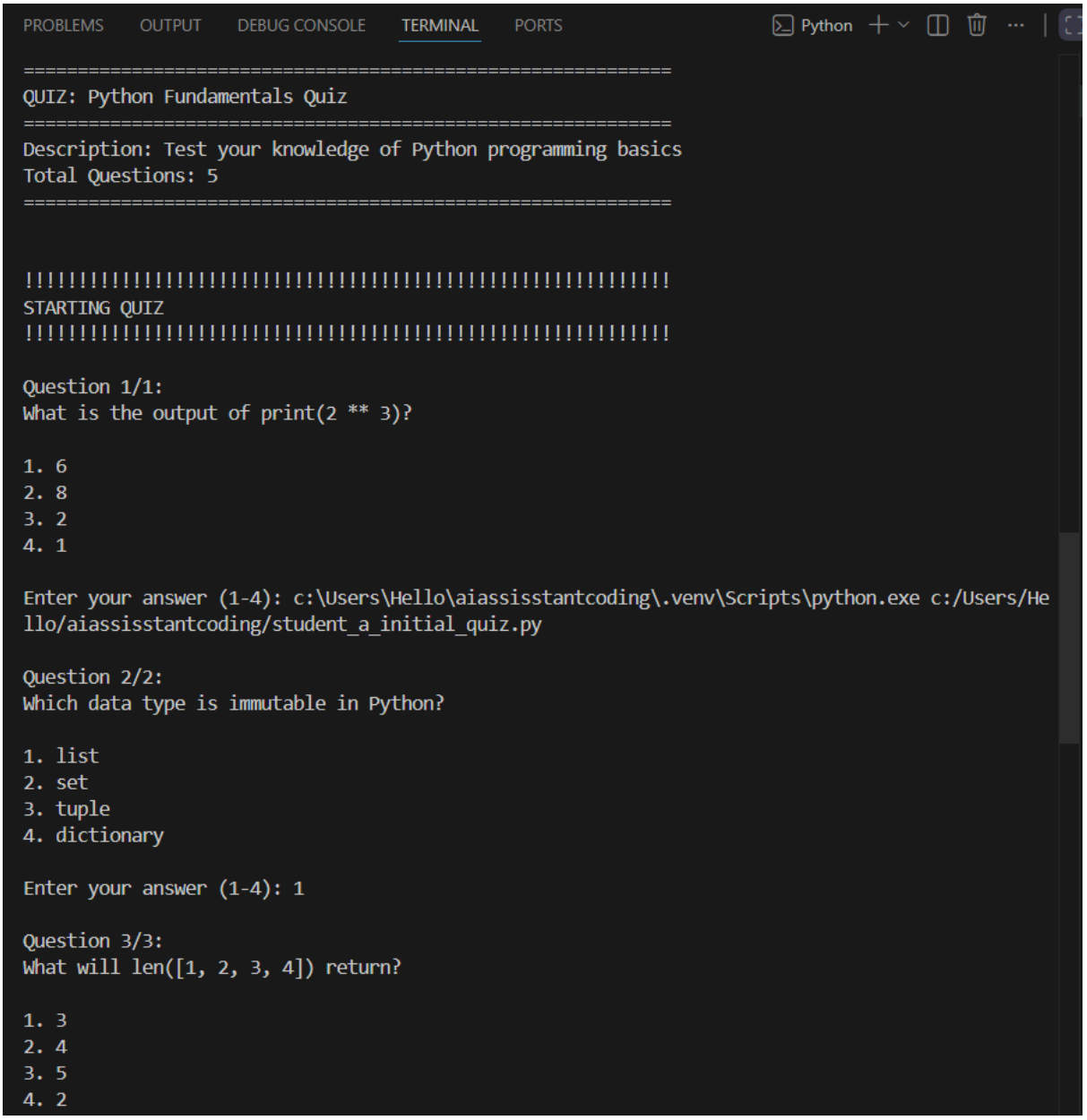
```python
def calculate_score(quiz, user_answers):
    """
    Calculate the total score based on correct answers.

    Args:
        quiz (dict): The quiz dictionary containing questions
        user_answers (list): List of user's answers (as option numbers)

    Returns:
        int: Number of correct answers
    """
    correct_count = 0

    for question, answer in zip(quiz['questions'], user_answers):
        try:
            answer_index = int(answer) - 1
            selected_option = question['options'][answer_index]
            if selected_option == question['correct_answer']:
                correct_count += 1
        except (ValueError, IndexError):
            pass

    return correct_count


def calculate_percentage(correct_count, total_questions):
    """
    Calculate percentage score.

    Args:
        correct_count (int): Number of correct answers
        total_questions (int): Total number of questions

    Returns:
        float: Percentage score (0-100)
    """
```

student_a_initial_quiz.py > ...

```
def run_quiz(quiz):
        list: List of user answers
    """
    user_answers = []

    print("\n" + "!"*60)
    print("STARTING QUIZ")
    print("!"*60 + "\n")

    for i, question in enumerate(quiz['questions'], 1):
        display_question(question, i)

        # Get answer
        answer = get_user_answer(len(question['options']))
        user_answers.append(answer)

        print()

    return user_answers


def display_results(quiz, user_answers):
    """
    Display quiz results with final score.

    Args:
        quiz (dict): The quiz dictionary
        user_answers (list): List of user's answers
    """
    correct_count = calculate_score(quiz, user_answers)
    total_questions = len(quiz['questions'])
    percentage = calculate_percentage(correct_count, total_questions)

    print("\n" + "="*60)
    print("QUIZ COMPLETED!")
    print("="*60)

```

```python
student_a_initial_quiz.py > ...
def display_results(quiz, user_answers):
                selected_option = question['options'][answer_index]
                is_correct = selected_option == question['correct_answer']
                status = "✓" if is_correct else "X"
                print(f"Q{i}: {answer} {status}")
            except (ValueError, IndexError):
                print(f"Q{i}: {answer} X")

    # Display final score
    print("\n" + "="*60)
    print("FINAL SCORE")
    print("="*60)
    print(f"Correct Answers: {correct_count}/{total_questions}")
    print(f"Percentage: {percentage}%")
    print("="*60 + "\n")


def main():
    """Main function to run the quiz program."""
    # Create the quiz
    quiz = create_quiz()

    # Display quiz information
    display_quiz_info(quiz)

    # Run the quiz
    user_answers = run_quiz(quiz)

    # Display results
    display_results(quiz, user_answers)

    print("Thank you for taking the quiz!")


if __name__ == "__main__":
    main()
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
==========
QUIZ: Python Fundamentals Quiz
==========
Description: Test your knowledge of Python programming basics
Total Questions: 5
==========
!!
STARTING QUIZ
!!
Question 1/1:
What is the output of print(2 ** 3)?
1. 6
2. 8
3. 2
4. 1
Enter your answer (1-4): c:\Users\Hello\aiassisstantcoding\.venv\Scripts\python.exe c:/Users/He
llo/aiassisstantcoding/student_a_initial_quiz.py
Question 2/2:
Which data type is immutable in Python?
1. list
2. set
3. tuple
4. dictionary
Enter your answer (1-4): 1
Question 3/3:
What will len([1, 2, 3, 4]) return?
1. 3
2. 4
3. 5
4. 2

```
How do you create a function in Python?

1. function myFunc():
2. def myFunc():
3. func myFunc():
4. define myFunc():

Enter your answer (1-4): 3

Question 5/5:
What is the correct way to create a list in Python?

1. {1, 2, 3}
2. [1, 2, 3]
3. (1, 2, 3)
4. list 1, 2, 3

Enter your answer (1-4): 1


============================================================
QUIZ COMPLETED!
============================================================

Your Answers:
Q1: c:\Users\Hello\aiassisstantcoding\.venv\Scripts\python.exe c:/Users/Hello/aiassissta
g/student_a_initial_quiz.py X
Q2: 1 X
Q3: 3 X
Q4: 3 X
Q5: 1 X

============================================================
FINAL SCORE
============================================================
Correct Answers: 0/5
Percentage: 0.0%
============================================================

Thank you for taking the quiz!
PS C:\Users\Hello\aiassisstantcoding>
```

Justification:

- Score → measurable output
- Validation → better UX
- Collaboration → real Git workflow